Personalized Networking Assistant

AI-Powered Networking Conversation Generator

Internship Project Documentation

Team Members

Team Lead

Nakka Kirthi	kirthinakka@gmail.com

Members

Suvarna Rayudu	rayudusuvarna0775@gmail.com
Neeraj Rathnakar Chappidi	cmneerajr@gmail.com
Amulya Satya Mida Gadi	queenamulyagadi@gmail.com
Nadimpalli Rani	raninadimpalli756@gmail.com

Technology Stack	Streamlit, FastAPI, DistilBERT, GPT-2, Wikipedia API
Document Type	Project Documentation Report
Version	1.0

Table of Contents

1. Project Overview

The PersonalizedNetworkingAssistant is an AI-powered web application designed to help users initiate meaningful conversations at professional networking events. The application analyzes an event description using Natural Language Processing (NLP), extracts the primary discussion themes, verifies those themes using Wikipedia, and generates personalized conversation starters based on the user's stated interests.

The project combines multiple AI technologies — DistilBERT for theme extraction, GPT-2 for natural language generation, FastAPI for the backend REST API, and Streamlit for the interactive frontend — to provide an intelligent, end-to-end networking assistant. A key design goal was reliability: the application is built so that it always produces a usable result, automatically falling back to deterministic rule-based logic if an AI model or external APIis temporarily unavailable.

2. Project Objectives

Theprimary objectivesofthis projectare to:

- Develop an AI-powered networking assistant that runs end-to-end locally.
- Extract the most relevant topics/themes from a free-text event description.
- Verify extracted topics against Wikipedia for factual grounding.
- Generate natural, human-like conversation starters tailored to user interests.
- Provide a simple, interactive web interface for non-technical users.
- Persist conversation history and user feedback locally as JSON.
- Expose all functionality through a documented REST API (FastAPI + Swagger UI).
- Validate correctness through automated unit and integration testing (Pytest + HTTPX).

3. Features

AI Event Analysis

- Extracts the dominant theme(s) from an event description.
- Uses Hugging Face DistilBERT (zero-shot classification) as the primary method.
- Falls back to keyword-frequency extraction if the model is unavailable.

Fact Verification

- Verifies the extracted theme using the Wikipedia API.
- Returns a short, citable factual summary plus the source URL.
- Handles ambiguous topics and missing articles gracefully.

AI Conversation Generation

- Generates natural-sounding, context-aware conversation starters.
- Uses GPT-2 (small) via the Hugging Face text-generation pipeline.
- Falls back to a curated template bank if GPT-2 is unavailable or its output is degenerate.

Conversation History

- Stores every generated conversation request and result.
- Uses local JSON storage (data/history.json) — no external database required.

Feedback Collection

- Allows users to like or dislike individual conversation starters.
- Saves feedback locally (data/feedback.json) for later review.

REST API

- FastAPI backend with full request/response validation via Pydantic.
- Interactive Swagger documentation available at /docs.

Interactive Frontend

- Streamlit-based dashboard for event input, generation, history, and feedback.
- Live indicator of whether AI models or fallback logic are currently active.

Automated Testing

- Unit tests for every service module using Pytest.
- API-level integration tests using FastAPI's TestClient (built on HTTPX).

4. Technologies Used

Technology	Purpose
Python 3.11+	Core programming language
FastAPI	Backend REST API framework
Streamlit	Frontend web dashboard
Hugging Face Transformers	AI model pipelines (zero-shot, text generation)
DistilBERT (NLI variant)	Theme / topic extraction from event descriptions
GPT-2 (small)	Natural-language conversation starter generation
Wikipedia API (wikipedia package)	Fact verification and summary retrieval
Pytest	Unit and integration testing framework
HTTPX (via FastAPI TestClient)	API endpoint testing
Pydantic	Request/response schema validation
JSON (local files)	Lightweight data persistence (history, feedback)
Git	Version control

5. System Architecture

Thesystem follows alayeredarchitecture:aStreamlit frontend communicates with a FastAPI backend over HTTP, the backend orchestrates three independent AI/data services, and results are persisted to local JSON storage.

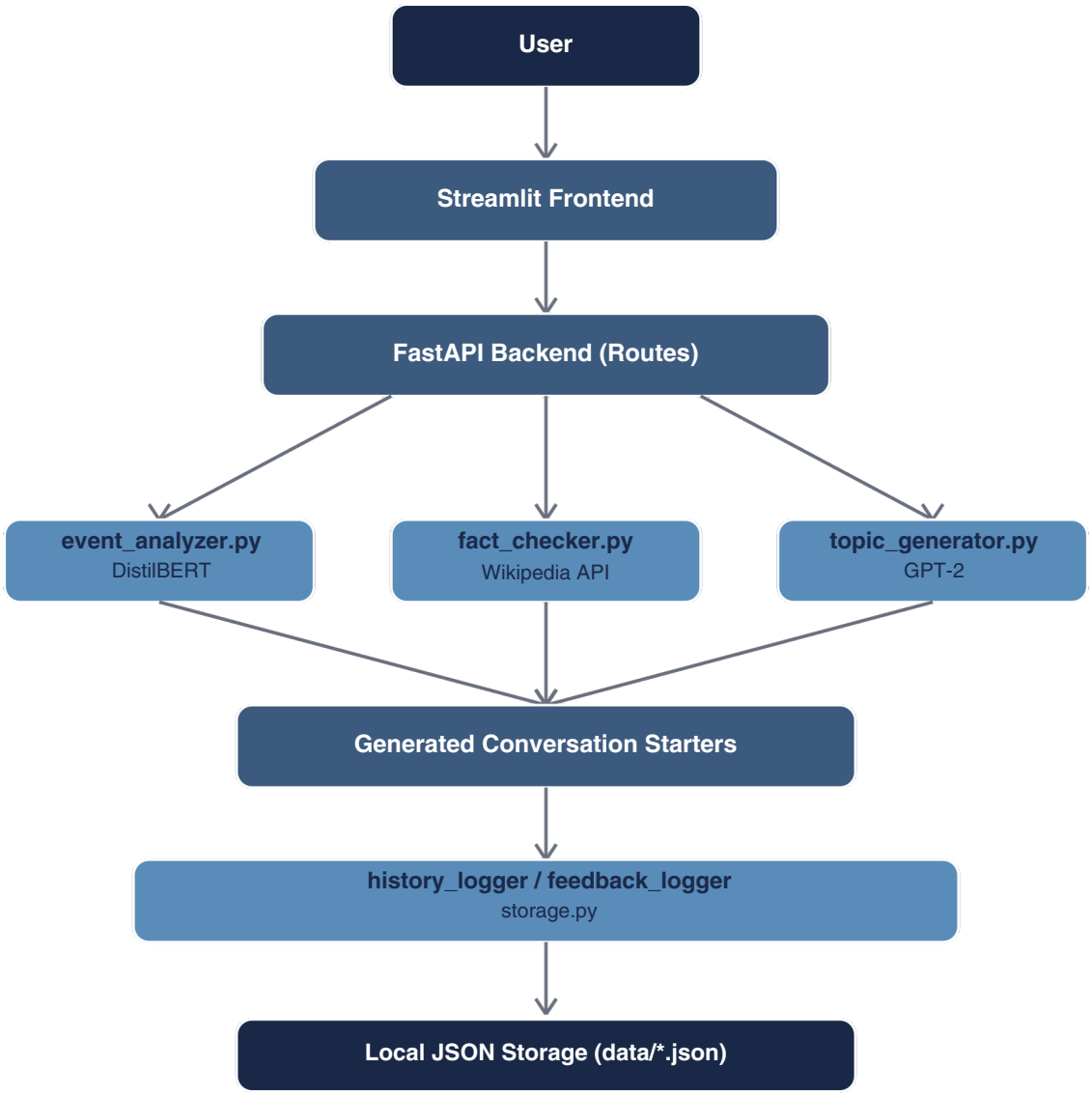

Figure 1: High-level system architecture and data flow.

6. Project Workflow

Step 1— User Input

The user enters an event description and, optionally, their personal interests in the Streamlit UI.

Step 2 — Request Received

The Streamlit frontend sends a POST request to the FastAPI backend's /generate-conversation endpoint.

Step 3 — Theme Extraction

event_analyzer.py uses DistilBERT zero-shot classification to identify the event's dominant theme(s).

Step 4 — Fact Verification

fact_checker.py looks up the top theme on Wikipedia and retrieves a short, verified factual summary.

Step 5 — Conversation Generation

topic_generator.py uses GPT-2 to generate personalized conversation starters that weave together the theme, the user's interests, and the verified fact.

Step 6 — Response Displayed

The generated conversation starters, detected themes, and fact-check result are displayed to the user.

Step 7 — History Logging

The complete result is automatically stored in data/history.json via storage.py.

Step 8 — Feedback Capture

If the user likes or dislikes a generated prompt, that action is stored in data/feedback.json.

7. Folder Structure

The project followsamodularstructure that separates API routes, business-logic services, data models, and the frontend into clearly defined layers.

```
networking-assistant/
|-- README.md
|-- .gitignore
|-- backend/

|   |-- requirements.txt
|   |-- pytest.ini
|   |-- app/
|   |   |-- main.py                      (FastAPI app entrypoint)
|   |   |-- routes/
|   |   |   |-- analyze_event.py         (POST /analyze-event)
|   |   |   |-- fact_check.py            (POST /fact-check)
|   |   |   |-- generate_conversation.py (POST /generate-conversation)
|   |   |   |-- feedback.py              (POST/GET /feedback, GET /history)
|   |   |-- services/
|   |   |   |-- model_loader.py     (lazy DistilBERT / GPT-2 loading + fallback)
|   |   |   |-- event_analyzer.py   (theme extraction)
|   |   |   |-- topic_generator.py (conversation generation)
|   |   |   |-- fact_checker.py     (Wikipedia verification)
|   |   |   |-- storage.py          (history / feedback JSON persistence)
|   |   |-- models/
|   |       |-- schemas.py (Pydantic request / response models)
|   |-- tests/
|       |-- conftest.py
|       |-- test_event_analyzer.py
|       |-- test_fact_checker.py
|       |-- test_topic_generator.py
|       |-- test_storage.py
|       |-- test_routes.py
|-- frontend/
|   |-- streamlit_app.py   (Streamlit UI: generate / history / feedback pages)
|   |-- requirements.txt
|-- data/
    |-- history.json
    |-- feedback.json
```

8. API Endpoints

Allendpoints areimplementedin FastAPI and are fully documented and testable through the interactive Swagger UI at **http://127.0.0.1:8000/docs**.

Method	Endpoint	Description
GET	/	Root endpoint — basic service info
GET	/health	Health check + AI model load status
POST	/analyze-event	Extracts theme(s) from an event description
POST	/fact-check	Verifies a topic against Wikipedia
POST	/generate-conversation	Full pipeline: analyze → fact-check → generate (+ logs history)
POST	/feedback	Logs a like / dislike / used feedback action
GET	/history	Returns recent conversation generation history
GET	/feedback	Returns recent feedback entries

9. Installation

1. Clone or extract the project

```
git clone https://github.com/SuvarnaRayudu/PersonalizedNetworkingAssistant
cd networking-assistant
```

2. Create a virtual environment

```
python3 -m venv venv
```

3. Activate the virtual environment

```
# macOS / Linux
source venv/bin/activate

# Windows
venv\Scripts\activate
```

4. Install dependencies

```
pip install --upgrade pip
pip install -r backend/requirements.txt
pip install -r frontend/requirements.txt
```

Note: torch and transformers are the heaviest dependencies. If they are not installed or fail to download model weights, the application automatically switches to fallback logic and continues to function end-to-end (see Section 13).

5. Initialize Git version control

```
git init
git add .
git commit -m "Initial commit: Personalized Networking Assistant"
```

10. Running the Project

Backend(FastAPI)

```
cd backend
uvicorn app.main:app --reload --port 8000
```

API root: http://127.0.0.1:8000 | Swagger UI: http://127.0.0.1:8000/docs

Frontend (Streamlit)

```
cd frontend
streamlit run streamlit_app.py
```

Application: http://localhost:8501. Run this in a second terminal while the backend is still running.

11. Testing

Runthefullautomatedtest suite with:

```
cd backend
pytest
```

The suite covers unit tests for event_analyzer, topic_generator, fact_checker, and storage, as well as integration tests for every API route using FastAPI's TestClient (built on HTTPX). Manual end-to-end validation was additionally performed through the Streamlit UI and the Swagger UI at http://127.0.0.1:8000/docs.

Actual result from the project's test run:

```
================================================
38 passed in 1.29s
================================================
```

Test coverage by file: test_event_analyzer.py (5), test_fact_checker.py (5), test_routes.py (15), test_storage.py (7), test_topic_generator.py (6).

12. Sample Execution

Input

Event Description: "Join us for a Tech Networking Mixer focused on artificial intelligence, startups, and cloud computing. Meet founders, engineers, and investors over drinks."
User Interests: machine learning, venture capital

Detected Themes

Artificial Intelligence • Startups • Cloud Computing

Wikipedia Summary

"Artificial intelligence is the capability of computational systems to perform tasks typically associated with human intelligence..."

Generated Conversation Starters

-
- Since you're into machine learning, have you found any overlap with artificial intelligence here?
- What got you into venture capital, and how does it connect with what's happening in AI right now?
  I'm curious — what's the most interesting thing you've worked on recently in cloud computing?

13. Demo Video

A complete demonstration of the Personalized Networking Assistant, including theme extraction, fact verification, and conversation generation, can be viewed using the link below.

Google Drive Demo Video:

https://drive.google.com/file/u/1/d/1RGeQ-54l_jToiFVXWb3lfxoC0noTk3Su/view?usp=sharing

Scan to watch the demo video

Source Code Repository:

https://github.com/SuvarnaRayudu/PersonalizedNetworkingAssistant

Scan for Source Code

14. Reliability and Fallback Design

Downloading and running real transformer models (DistilBERT and GPT-2) requires internet access and several hundred megabytes of disk space, and external services such as Wikipedia can occasionally be unreachable. To ensure the application is always demo-able and never crashes due to an external dependency, every AI/network call in this project is wrapped in fault-tolerant logic:

- If DistilBERT fails to load, event_analyzer.py automatically falls back to a keyword-frequency based theme extractor.
- If GPT-2 fails to load, or its output is too short or repetitive, topic_generator.py automatically falls back to a curated, natural-sounding template bank.
- If Wikipedia is unreachable, ambiguous, or has no matching article, fact_checker.py returns a structured "unverified" result instead of raising an error.
- The /health endpoint reports in real time whether each model is loaded, failed, or running on fallback logic, and the Streamlit sidebar surfaces this status to the user.

This means the complete pipeline — theme extraction, fact-checking, and conversation generation — remains fully functional and testable even in constrained or offline environments, while seamlessly using the real AI models whenever they are available.

15. Future Enhancements

- User authentication and login.
- Cloud database integration (replacing local JSON storage).
- Multi-language support for event descriptions and generated prompts.
- Real-time networking recommendations based on attendee interest overlap.
- LinkedIn profile integration for richer personalization.
- Resume analysis to refine suggested talking points.
- Speech-to-text input for hands-free conversation generation.
- AI voice assistant mode for live use during events.
- Containerized deployment with Docker.
- Cloud deployment on AWS, Azure, or GCP.

16. Learning Outcomes

Thisproject provided hands-onexperiencein:

- Natural Language Processing and transformer-based AI models.
- Zero-shot classification and text-generation pipelines with Hugging Face Transformers.
- REST API design and development using FastAPI, including request/response validation with Pydantic.
- Frontend development using Streamlit, including multi-page apps and live API integration.
- Designing fault-tolerant systems that degrade gracefully when external dependencies fail.
- Local JSON-based data persistence patterns.
- Automated software testing with Pytest and FastAPI's HTTPX-based TestClient.
- Version control using Git.
- Modular, service-oriented software architecture.

17. Conclusion

The Personalized Networking Assistant successfully demonstrates the integration of modern AI technologies into a practical, real-world networking solution. By combining DistilBERT for event analysis, Wikipedia for fact verification, GPT-2 for natural language generation, FastAPI for backend services, and Streamlit for an interactive frontend, the application provides users with context-aware conversation starters tailored to their interests.

The project follows a modular architecture, includes a comprehensive automated test suite (38 passing tests across unit and integration levels), implements local data persistence for history and feedback, and is explicitly engineered for reliability through graceful fallback behavior. Together, these qualities make it a complete, scalable, and industry-oriented AI application suitable for internship evaluation and future enhancement.